



Pontifícia
Universidade
Católica do
Rio de Janeiro

DEI
DEPARTAMENTO
DE ENGENHARIA
INDUSTRIAL



ENG 4560 – Projeto Integrado VI: Distribuição Física | 2026.1

Aula 7B: Heurísticas construtivas

Prof. Marcello Congro

marcellocongro@puc-rio.br



CTC
CENTRO TÉCNICO CIENTÍFICO / PUC-Rio

Onde estamos no curso?

- Nesta aula, daremos continuidade ao problema de roteirização. Após a Sprint 1, em que implementamos e analisamos modelos exatos para o CVRP, passamos a discutir estratégias heurísticas construtivas.



A partir de agora, vamos discutir algoritmos mais rápidos capazes de gerar soluções viáveis para as instâncias em que os métodos exatos se tornam caros ou lentos sob o ponto de vista computacional.

- Na **Aula 2B**, transformamos a base operacional em uma instância formal do problema de roteirização (nós, clientes, distâncias, demandas, tempos, etc);
- Na **Aula 3**, escrevemos a primeira formulação matemática do CVRP, ainda em versão simplificada, focando nas variáveis de decisão e restrições fundamentais;
- Na **Aula 4**, tornamos o modelo estruturalmente mais correto e operacionalmente mais realista, incorporando a eliminação de subtours (MTZ) e considerando frota heterogênea.



<https://prologtransportes.com.br/>

Onde estamos no curso? (cont.)

- A experiência da Sprint 1 mostrou que:



Modelos exatos são conceitualmente poderosos, mas podem se tornar computacionalmente caros à medida que o problema cresce.

- A partir desta aula, vamos discutir uma nova pergunta:



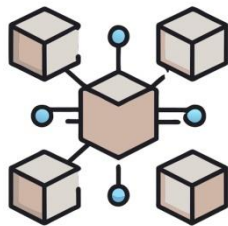
Como obter soluções boas e rápidas quando o método exato já não é alternativa mais eficiente para fins operacionais?



<https://prologtransportes.com.br/>

Por que sair dos métodos exatos?

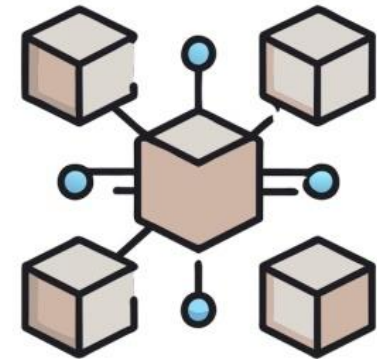
- Os métodos exatos são fundamentais porque permitem formular corretamente o problema, compreender sua estrutura e, em instâncias menores, encontrar soluções ótimas com prova de otimalidade.
- Entretanto, o CVRP é um problema combinatório cuja complexidade cresce rapidamente com o número de clientes e com o “enriquecimento” do modelo (mais restrições, como tipos de veículos, conectividade global, tempos, capacidade e outras regras);
- Do ponto de vista gerencial, isto cria um **dilema real**
 - ❖ Uma solução matematicamente ótima pode levar tempo demais para ser obtida;
 - ❖ Isso se torna pouco útil em contextos operacionais que exigem resposta rápida;
 - ❖ Nesse cenário, surgem as **heurísticas: métodos que não garantem a otimalidade, mas entregam soluções viáveis em tempo reduzido**, o que muitas vezes é o que a operação precisa.



Métodos exatos buscam provar o melhor; heurísticas buscam entregar rápido uma solução suficientemente boa!

O que é uma heurística?

- É um procedimento sistemático que constrói ou melhora soluções viáveis sem a pretensão de provar que a melhor solução global foi encontrada;
- Diferentemente dos métodos exatos, a heurística não busca explorar exhaustivamente o espaço de soluções nem estabelecer limites formais de otimalidade;
- Ao invés disso, utiliza regras de decisão inteligentes, geralmente inspiradas em características estruturais do problema, para chegar rapidamente a soluções úteis;
 - ❖ Isso não significa “fazer de qualquer jeito”: a heurística incorpora lógica operacional, aproveita a estrutura dos dados e pode produzir resultados competitivos na prática.



Construir x melhorar

- Existem duas grandes famílias de estratégias (heurísticas):
 - ❖ **Heurísticas construtivas**: métodos que constroem uma solução do zero, definindo progressivamente quais clientes serão atendidos e em qual ordem;
 - ❖ **Heurísticas de melhoria**: também chamados de métodos de busca local, que partem de uma solução já existente e tentam aprimorá-la por meio de pequenas modificações.
- Na aula de hoje, vamos focar na primeira família: como construir uma solução inicial viável para o problema CVRP
 - ❖ Antes de *melhorar uma rota*, é preciso ser capaz de **gerar uma rota**.
 - ❖ Antes de *refinar uma solução*, é preciso ter uma **solução inicial sobre a qual operar**.
- Exemplos de heurísticas construtivas: Nearest Neighbor (Vizinho Mais Próximo) e Clarke-Wright Savings.

Heurísticas
construtivas

Heurísticas
de busca
local

Construir x melhorar (cont.)

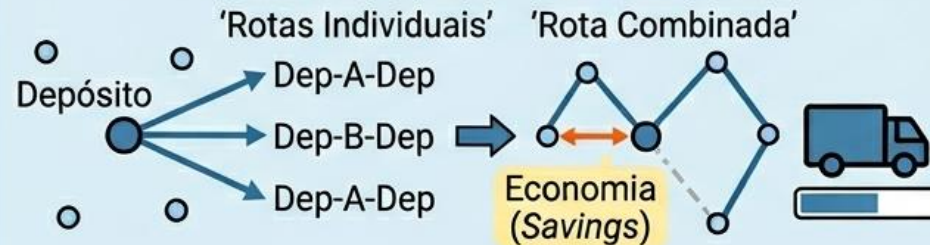


HEURÍSTICAS CONSTRUTIVAS (MÉTODOS DO ZERO)

Exemplo: Nearest Neighbor



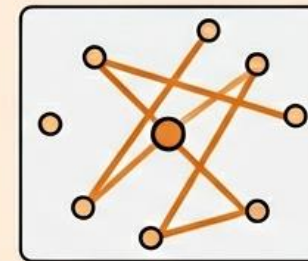
Exemplo: Clarke-Wright Savings



Foco de hoje: Gerar uma solução inicial viável para o CVRP.

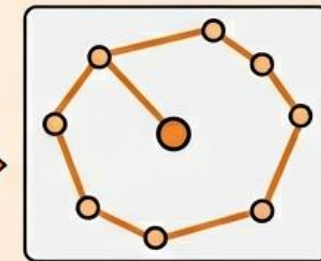


HEURÍSTICAS DE MELHORIA (BUSCA LOCAL)

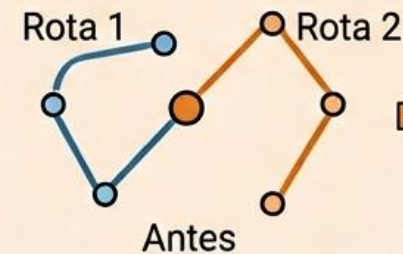


Solução com Cruzamentos

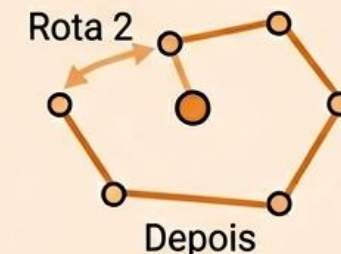
Pequenas Modificações
(ex: 2-opt)



Solução Refinada



Antes



Depois

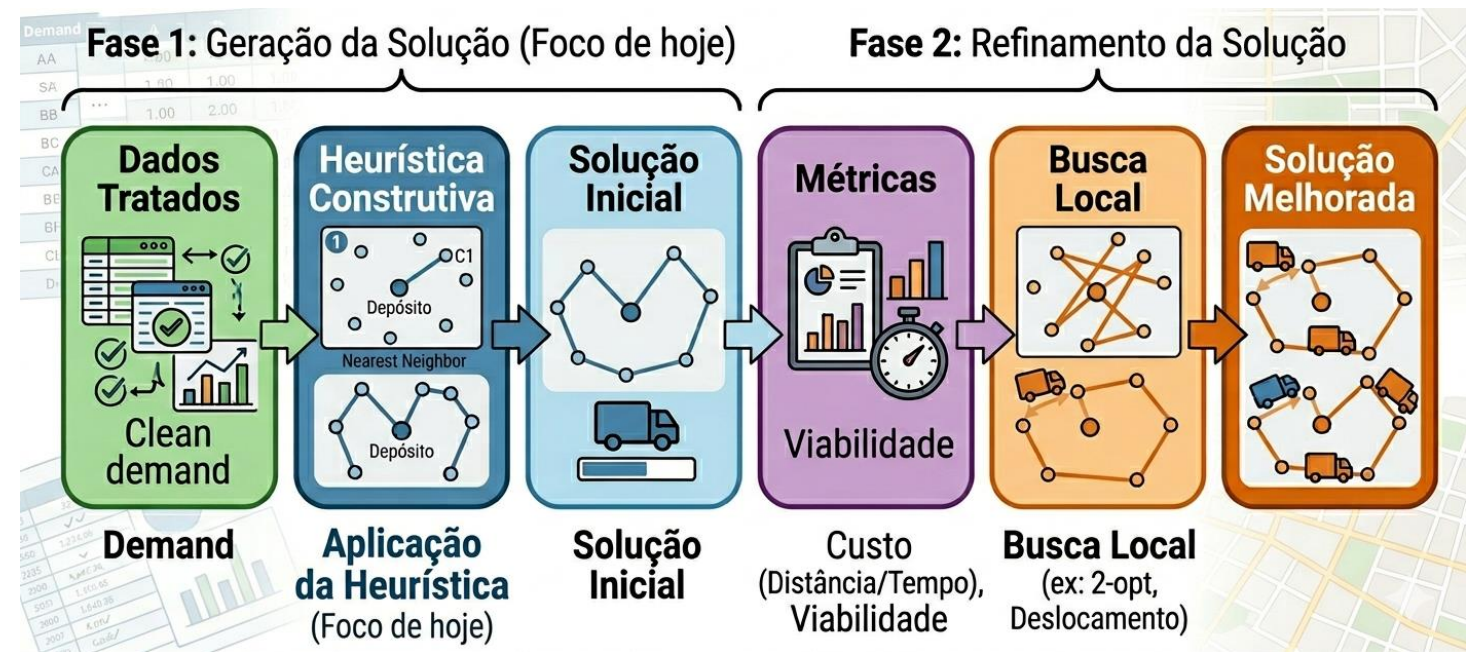
Refinar a solução existente para reduzir custos e melhorar a eficiência.

Pipeline da solução

- Etapas para obtenção da solução:

1. Construimos a solução inicial viável por meio de uma heurística construtiva;
2. Avaliamos essa solução sob diferentes métricas (custo total, número de veículos utilizados, distância percorrida, tempo computacional);
3. Aplicamos estratégias de melhoria local sobre essas soluções, buscando reduzir custos ou reorganizar rotas.

- Raramente um problema real de logística é resolvido em um único passo;
- O mais comum é trabalhar de forma iterativa: construir, avaliar, ajustar e melhorar. É exatamente essa lógica que será incorporada nas atividades das Sprints 2 e 3.



A solução inicial importa!

- Por que a qualidade da solução inicial importa?
 - ❖ Uma solução inicial ruim pode comprometer o restante do processo;
 - ❖ Se uma heurística construtiva gera rotas pouco eficientes, com veículos mal aproveitados ou distâncias excessivas, a etapa de melhoria posterior terá muito mais trabalho para recuperar desempenho;
 - ❖ Uma solução inicial bem construída reduz o esforço de refinamento e pode, inclusive, já ser suficientemente boa para determinadas aplicações operacionais.



- Qual heurística gera uma solução inicial mais promissora?
- Qual delas respeita melhor a lógica do problema?
- Qual delas oferece melhor relação entre simplicidade, qualidade e tempo computacional?

Métodos gulosos

- O que significa ser “guloso”?
 - ❖ É um método que toma decisões localmente atraentes em cada passo, sem reconsiderar globalmente todas as consequências futuras;
 - ❖ Em outras palavras, o método escolhe o que parece melhor naquele momento com base em regra simples e imediata;
- Entretanto, decisões localmente boas nem sempre conduzem à melhor solução global. Esse é o “preço” da simplicidade. Ainda assim, são muito valiosos como ponto de partida e baseline para comparação/entendimento do problema.



Decisão localmente boa não implica em solução globalmente boa!

Nearest Neighbor (Vizinho Mais Próximo)

- Heurística do vizinho mais próximo
 - ❖ A ideia é começar no depósito e, a cada passo, escolher o cliente viável mais próximo do ponto atual;
 - ❖ O veículo segue acumulando clientes enquanto houver capacidade disponível e, quando não for mais possível continuar de forma viável, retorna ao depósito e inicia-se uma nova rota com outro veículo.
- Sob o ponto de vista computacional, é um método extremamente rápido e intuitivo;
- Sob o ponto de vista logístico, captura uma lógica simples, mas seu alcance é limitado: por olhar apenas o próximo passo, ele ignora o efeito global da decisão sobre o restante da malha de entregas.



Heurística do Vizinho Mais Próximo

Nearest Neighbor (Vizinho Mais Próximo) (cont.)

- Como o algoritmo funciona na prática?
 - ❖ Seleciona-se um veículo disponível e inicia-se a rota a partir do depósito;
 - ❖ Identifica-se o conjunto de clientes ainda não atendidos que podem ser inseridos na rota sem violar a capacidade do veículo;
 - ❖ Dentre esses clientes viáveis, escolhe-se aquele que está a menor distância do ponto atual;
 - ❖ O cliente é então inserido na rota, sua demanda é incorporada à carga acumulada e o processo se repete.

- Quando não existir cliente viável adicional, a rota é encerrada com o retorno ao depósito;

- Se ainda houver clientes não atendidos, inicia-se uma nova rota. Esse procedimento se repete até que todos os clientes tenham sido alocados.



Não colocamos MTZ nem outra restrição de subtour porque, nas heurísticas construtivas, a rota é construída diretamente como sequência conectada ao depósito.

Nearest Neighbor (Vizinho Mais Próximo) (cont.)

▪ Limitações

- ❖ Ao sempre privilegiar o cliente mais próximo do ponto atual, o método pode tomar decisões que parecem boas no curto prazo, mas que dificultam o restante do planejamento;
 - ❖ É comum, por exemplo, que deixe clientes mais distantes ou geometricamente inconvenientes para o final, o que pode resultar em rotas longas, desbalanceadas ou mal distribuídas.
-
- O método não incorpora explicitamente uma noção de economia global entre combinações de clientes;
 - Ou seja, ele responde bem à pergunta **“Onde fica o cliente mais próximo?”**, mas não responde necessariamente à pergunta **“Qual é a melhor forma de agrupar clientes em rotas?”**
 - Apesar de ser uma heurística válida e útil, costuma funcionar melhor como baseline comparativo.



Clarke-Wright Savings

- O algoritmo de Clarke-Wright Savings faz com que cada cliente seja atendido por uma rota exclusiva que sai do depósito, visita o cliente e retorna ao depósito. Essa solução é viável, mas normalmente muito ineficiente;
- A seguir, o algoritmo avalia a possibilidade de unir pares de rotas, conectando clientes entre si, sempre que essa função gerar economia e respeitar as restrições do problema;
- Ao invés de fazer duas viagens separadas “depósito – i – depósito” e “depósito – j – depósito”, pode ser melhor fazer uma única sequência que conecte os clientes, reduzindo a distância total percorrida.
- Seu diferencial não está em escolher diretamente o cliente mais próximo, ms em avaliar quanto se economiza ao conectar clientes entre si ao invés de tratá-los como rotas independentes a partir do depósito.



Clarke-Wright Savings (cont.)

- Economia (“*saving*”)
 - ❖ Para um par de clientes i e j , define-se a economia associada à sua conexão por:

$$S_{ij} = c_{i0} + c_{j0} - c_{ij}$$

c_{i0} : custo de ir do cliente i ao depósito
 c_{j0} : custo de ir do cliente j ao depósito
 c_{ij} : custo de ligar diretamente i a j .

- S_{ij} mede **quanto se economiza** ao substituir duas ligações independentes com o depósito por uma ligação direta entre dois clientes.
- Quanto maior o valor de S_{ij} , mais vantajoso tende a ser unir esses clientes em uma mesma rota, desde que as demais restrições permaneçam satisfeitas.



O saving não mede distância absoluta, mas sim o ganho relativo de juntar clientes!

Clarke-Wright Savings (cont.)

- Economia (“*saving*”)
 - ❖ Para um par de clientes i e j , define-se a economia associada à sua conexão por:

$$S_{ij} = c_{i0} + c_{0j} - c_{ij}$$

c_{i0} : custo de ir do cliente i ao depósito
 c_{j0} : custo de ir do cliente j ao depósito
 c_{ij} : custo de ligar diretamente i a j .

- S_{ij} mede **quanto se economiza** ao substituir duas ligações independentes com o depósito por uma ligação direta entre dois clientes.
- Quanto maior o valor de S_{ij} , mais vantajoso tende a ser unir esses clientes em uma mesma rota, desde que as demais restrições permaneçam satisfeitas.



Logisticamente, se dois clientes estão geograficamente posicionados de modo que uma conexão direta entre eles substitui com vantagem dois retornos ao depósito, então a economia de S_{ij} será alta!



Isso sinaliza que pode ser operacionalmente interessante colocá-lo na mesma rota!

Clarke-Wright Savings (cont.)



É uma heurística construtiva gulosa orientada por economia de rota

- Como o algoritmo funciona na prática?
 - ❖ Cria-se uma solução inicial em que cada cliente forma uma rota própria;
 - ❖ Calcula-se o valor de S_{ij} para todos os pares de clientes candidatos;
 - ❖ Os *savings* são ordenados em ordem decrescente;
 - ❖ Algoritmo percorre essa lista e, para cada par, verifica se as rotas contendo i e j podem ser unidas sem violar a capacidade e sem gerar inconsistências estruturais.
- Se a fusão for viável: rotas são combinadas;
- Caso contrário: algoritmo passa ao próximo par;
- O processo continua até que não existam mais combinações vantajosas ou viáveis.



Clarke-Wright Savings (cont.)

- Restrições na fusão de rotas
 - ❖ Não basta que um par de clientes tenha *saving* elevado. Para que a fusão de rotas seja permitida, é necessário verificar condições estruturais e operacionais;
 - ❖ A soma das demandas dos clientes não pode exceder o limite de carga considerado;
 - ❖ É preciso garantir que a união preserve uma estrutura de rota coerente, sem duplicidade de clientes e sem criar ciclos inconsistentes.
- Mesmo sendo métodos construtivos rápidos, as heurísticas continuam submetidos às restrições do problema;
- Ao invés de resolver um modelo global inteiro, tomam decisões construtivas sucessivas guiadas por regras práticas.



Comparação entre heurísticas construtivas



Vizinho Mais Próximo

- ✓ Decisões locais e passo a passo
- ✓ Extremamente transparente
- ✓ Útil como baseline comparativo



Clarke–Wright

- ✓ Busca combinações económicas
- ✓ Soluções iniciais mais estruturadas
- ✓ Particularmente útil para malhas fixas

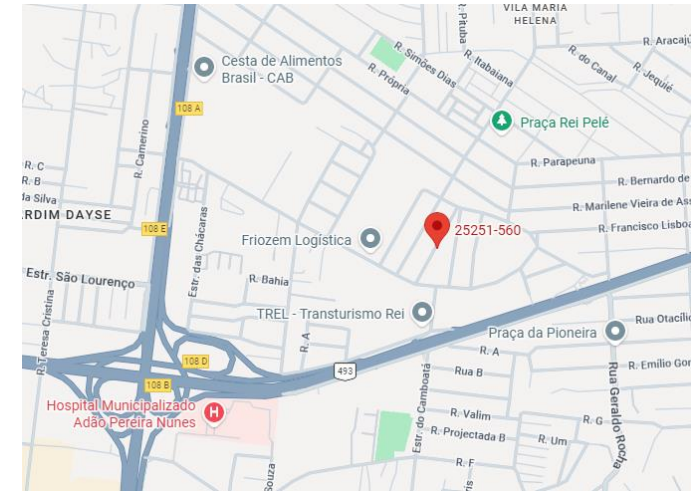
Heurísticas construtivas e rápidas, mas com bases distintas

O que vamos comparar na prática?

- Vamos comparar **criticamente** os algoritmos com base em métricas relevantes para o nosso problema prático;
- Entre as principais métricas, destacam-se:
 - ❖ **Custo total da solução;**
 - ❖ **Distância total percorrida;**
 - ❖ **Número de veículos utilizados;**
 - ❖ **Tempo computacional de execução;**
 - ❖ **Qualidade relativa da solução quando comparada ao modelo exato ou à melhor solução disponível.**
- Além disso, será importante observar como cada heurística responde às diferentes instâncias (C1-C4) já construídas previamente na Sprint 1;
- Uma estratégia pode funcionar muito bem em uma instância menor e perder desempenho em outra mais complexa. É justamente essa análise comparativa que transforma a implementação em uma atividade de Engenharia.

Características do sistema logístico

- O sistema logístico do projeto considerado nesta disciplina será caracterizado pelos seguintes parâmetros reais de operação:
 - ❖ Depósito (Centro de Distribuição – CD): **CEP 25251-560**
 - ✓ Este ponto será o local de origem e retorno de todas as rotas.
 - ❖ Tipos de veículos disponíveis:
 - ✓ Fiorino: Capacidade máxima **650 kg**, custo fixo diário **R\$ 250,00**;
 - ✓ VUC: Capacidade máxima **3000 kg**, custo fixo diário **R\$ 550,00**.
 - ❖ Custos e tempos adicionais:
 - ✓ Custo variável: R\$ 1,50/km (aplicável a todos os veículos);
 - ✓ Velocidade média: 40 km/h;
 - ✓ Tempo de atendimento por cliente: 15 min;
 - ✓ Jornada máxima diária: 8h (= 480 min).
- Esses parâmetros serão utilizados como entrada no problema, e, portanto, devem ser registrados e mantidos consistentes ao longo das análises.



Parte prática – Colab



Os alunos deverão:

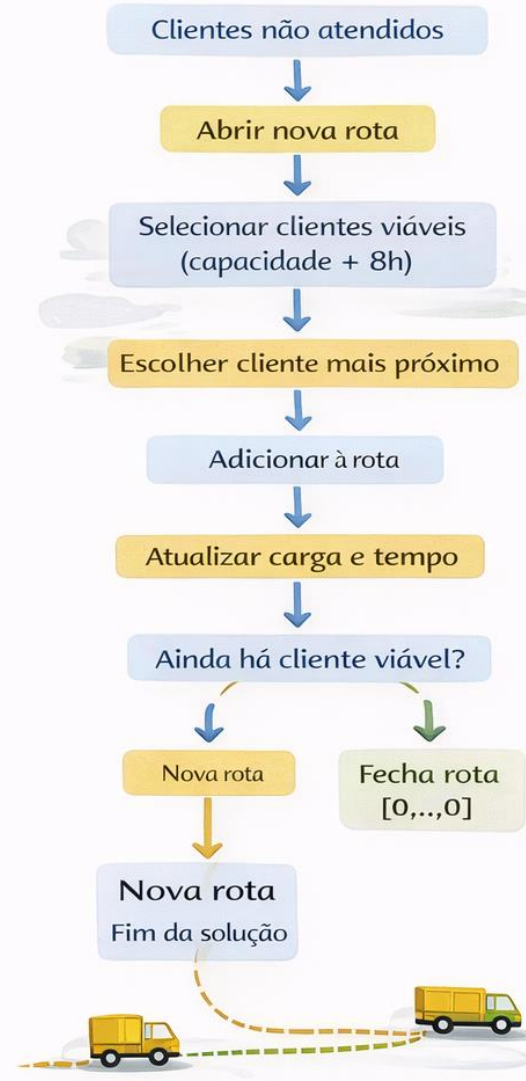
- ✓ carregar as instâncias já preparadas;
- ✓ implementar uma versão do vizinho mais próximo;
- ✓ calcular savings e ordenar pares de clientes;
- ✓ implementar a lógica de fusão de rotas do Clarke–Wright;
- ✓ comparar as soluções com base em métricas operacionais e computacionais.



Métodos construtivos tomam decisões práticas, mas respeitam limites do problema

Heurísticas construtivas

◆ Nearest Neighbor (crescimento sequencial)



◆ Clarke–Wright (consolidação de rotas)



Parte prática – Colab (cont.)



- Como as heurísticas estão sendo implementadas no Python?

❑ Nearest Neighbor (NN)

- ❖ Inicializa todos os clientes não atendidos;
- ❖ Abre uma nova rota a partir do depósito;
- ❖ Em cada passo:
 - Identifica **clientes viáveis**:
 - ✓ Respeitam capacidade;
 - ✓ Mantém a rota dentro de 8h (já considerando retorno ao depósito);
- ❖ Escolhe **o mais próximo do nó atual**;
- ❖ Repete até não haver mais cliente viável;
- ❖ Ao abrir cada rota:
 - Simula com **FIO** e **VUC**;
 - Calcula custo;
 - Escolhe o **melhor veículo**.

Crescimento sequencial da rota
(decisão local a cada cliente)

Parte prática – Colab (cont.)



- Como as heurísticas estão sendo implementadas no Python?

□ Clarke-Wright Savings

- ❖ Inicializa com uma rota por cliente: $[0, i, 0]$
- ❖ Calcula os *savings* $s_{ij} = d_{0i} + d_{0j} - d_{ij}$
- ❖ Ordena pares por maior economia
- ❖ Para cada par:
 - Verifica se estão em rotas diferentes e em extremidades;
 - Constrói a rota fundida candidata;
 - Testa a rota com:
 - FIO;
 - VUC.
 - Aceita a fusão se for viável (capacidade + tempo);
 - Escolhe o melhor veículo para a nova rota.

Consolidação de rotas
(decisão local a cada fusão)